Object-Oriented Programming in Java: Bank Account System

Comprehensive technical lab report detailing inheritance, encapsulation, method overriding, UML design, algorithm, implementation, and execution tracing.

1. PROBLEM STATEMENT / QUESTION

Design and implement an object-oriented **Bank Account Management System** in Java demonstrating **Hierarchical Inheritance**, **Encapsulation**, and **Polymorphism (Method Overriding)**.

- Create a superclass `Account` with attributes for account number, holder name, balance, and account type, along with operations for `deposit()`, `withdraw()`, `transfer()`, and `displayAccountDetails()`.
- Create a subclass `SavingsAccount` inheriting from `Account` that includes an `interestRate` and a specialized method `calculateInterest()`.
- Create a subclass `CurrentAccount` inheriting from `Account` that features an `overdraftLimit` and overrides `withdraw()` to facilitate overdraft protection.
- Provide a demonstration driver class `BankDemo` that creates accounts, displays before/after states, and tests deposits, withdrawals, interest accrual, and inter-account funds transfer.

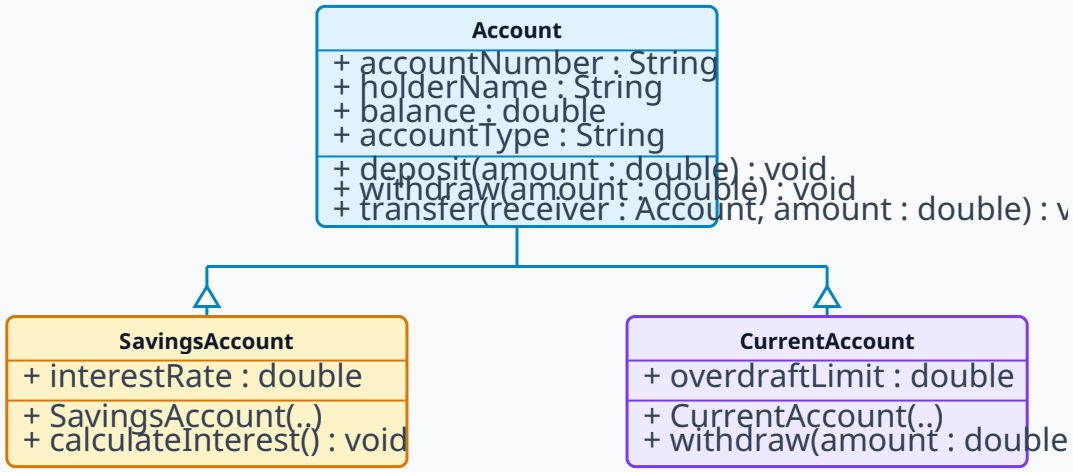
2. STEP-BY-STEP PROCESS

1. **Base Class Architecture (`Account`):** Declare fundamental instance fields: `accountNumber`, `holderName`, `balance`, and `accountType`. Implement parameterized constructor initializing these fields.
2. **Base Transaction Logic:**
   - `deposit(amount)`: Add transaction amount directly to `balance`.
   - `withdraw(amount)`: Validate availability ($amount \le balance$). Deduct if valid; otherwise alert insufficient balance.
   - `transfer(receiver, amount)`: Check sender balance, subtract from sender, and add to receiver object.
   - `displayAccountDetails()`: Print formatted account attributes.
3. **Derived Class 1 (`SavingsAccount`):** Extend `Account` using `super()`. Introduce `interestRate`. Add `calculateInterest()` which computes interest ($balance imes rate / 100$) and credits it to balance.
4. **Derived Class 2 (`CurrentAccount`):** Extend `Account` using `super()`. Introduce `overdraftLimit`. Override `withdraw(amount)` to allow withdrawals up to $(balance + overdraftLimit)$.
5. **Execution & Driver (`BankDemo`):** Instantiate accounts, log initial data, perform operations, credit interest, transfer funds, and display updated balances.

3. ALGORITHM

1. **Start**.
2. Define parent class `Account` with variables and deposit, withdrawal, transfer, and display methods.
3. Define `SavingsAccount extends Account`:
   - Add field `interestRate`.
   - Compute interest: $interest = (balance imes interestRate) / 100$.
   - Update: $balance = balance + interest$.
4. Define `CurrentAccount extends Account`:
   - Add field `overdraftLimit`.
   - Override `withdraw(amount)`: If $amount \le (balance + overdraftLimit)$, update $balance = balance - amount$; Else display limit exceeded.
5. In `main()`:
   - Initialize `savings("S101", "Bharath", 20000, 5)`.
   - Initialize `current("C201", "Bunny", 15000, 3000)`.
   - Print initial account profiles.
   - Deposit 2000 & withdraw 1000 from `savings`.
   - Deposit 3000 & withdraw 9000 from `current`.
   - Call `savings.calculateInterest()`.
   - Call `savings.transfer(current, 2000)`.
   - Print final account states and final balances.
6. **Stop**.

4. UML CLASS DIAGRAM



Inheritance Type: Hierarchical (Account is specialized into SavingsAccount and CurrentAccount)

5. JAVA SOURCE CODE

```java
// Bank Account System using OOP concepts
// Base class: common details and operations of every account
class Account {
    String accountNumber;
    String holderName;
    double balance;
    String accountType;

    // Constructor initializes account details
    Account(String number, String name, double balance, String type) {
        this.accountNumber = number;
        this.holderName = name;
        this.balance = balance;
        this.accountType = type;
    }

    // Adds the given amount to the account
    void deposit(double amount) {
        balance += amount;
    }

    // Withdraws only when enough balance is available
    void withdraw(double amount) {
        if (amount <= balance)
            balance -= amount;
        else
            System.out.println("Insufficient balance!");
    }

    // Transfers money from this account to another account
    void transfer(Account receiver, double amount) {
        if (amount <= balance) {
            balance -= amount;
            receiver.balance += amount;
            System.out.println("Transfer successful!");
        } else {
            System.out.println("Transfer failed!");
        }
    }
```

```
    // Displays basic account information
    void displayAccountDetails() {
        System.out.println("
Account Number : " + accountNumber);
        System.out.println("Holder Name    : " + holderName);
        System.out.println("Account Type   : " + accountType);
        System.out.println("Balance        : " + balance);
    }
}

// SavingsAccount inherits all features of Account
class SavingsAccount extends Account {
    double interestRate;

    // Constructor initializes savings account with interest rate
    SavingsAccount(String number, String name, double balance, double interestRate) {
        super(number, name, balance, "Savings");
        this.interestRate = interestRate;
    }

    // Calculates simple interest on current balance
    void calculateInterest() {
        double interest = balance * interestRate / 100;
        balance += interest;
        System.out.println("Interest Added : " + interest);
    }
}

// CurrentAccount inherits Account and supports overdraft
class CurrentAccount extends Account {
    double overdraftLimit;

    // Constructor initializes current account with overdraft limit
    CurrentAccount(String number, String name, double balance, double overdraftLimit) {
        super(number, name, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    // Overriding withdraw() to allow overdraft
    @Override
    void withdraw(double amount) {
        if (amount <= balance + overdraftLimit)
            balance -= amount;
        else
            System.out.println("Overdraft limit exceeded!");
    }
}

// Main class: creates objects and performs transactions
public class BankDemo {
    public static void main(String[] args) {
        // Creating Savings and Current account objects
        SavingsAccount savings = new SavingsAccount("S101", "Bharath", 20000, 5);
        CurrentAccount current = new CurrentAccount("C201", "Bunny", 15000, 3000);

        // Displaying initial account details
        System.out.println("===== BEFORE TRANSACTIONS =====");
        savings.displayAccountDetails();
        current.displayAccountDetails();

        // Performing deposit and withdrawal operations
        savings.deposit(2000);
        savings.withdraw(1000);
        current.deposit(3000);
        current.withdraw(9000);   // Uses overdraft facility
```

```
        // Adding interest to savings account
        savings.calculateInterest();

        // Transferring money from savings to current account
        savings.transfer(current, 2000);

        // Displaying final account details
        System.out.println("
===== AFTER TRANSACTIONS =====");
        savings.displayAccountDetails();
        current.displayAccountDetails();

        // Showing final balances
        System.out.println("
Final Savings Balance : " + savings.balance);
        System.out.println("Final Current Balance : " + current.balance);
    }
}
```

6. SAMPLE OUTPUT

```
===== BEFORE TRANSACTIONS =====

Account Number : S101
Holder Name : Bharath
Account Type : Savings
Balance : 20000.0

Account Number : C201
Holder Name : Bunny
Account Type : Current
Balance : 15000.0
Interest Added : 1050.0
Transfer successful!

===== AFTER TRANSACTIONS =====

Account Number : S101
Holder Name : Bharath
Account Type : Savings
Balance : 20050.0

Account Number : C201
Holder Name : Bunny
Account Type : Current
Balance : 11000.0

Final Savings Balance : 20050.0
Final Current Balance : 11000.0
```